Pandas & Data Analysis Basics

Zhenlu Qin

August, 2025

How to Use These Slides

Each tool is introduced with: **Purpose** → **Parameters (what you can pass)** → **Usage patterns** → **Examples**. Practice is easy and mirrors the demos.

Setup & Imports

Purpose: enable Pandas/NumPy and check versions.
Usage:

- `import pandas as pd`, `import numpy as np`
- `pd.__version__` returns string version.

```
import pandas as pd
import numpy as np

pd.__version__
np.__version__
```

pd.Series: Parameters & Usage

Purpose: 1D labeled array.
Signature (common args): `pd.Series(data=None, index=None, dtype=None, name=None)`
Parameters:

- `data`: list/array/ dict / scalar.
- `index`: list-like labels; for dict, default is dict keys.
- `dtype`: target dtype e.g. `"int64"`, `"float64"`, `"category"`.
- `name`: optional series name.

Usage patterns: construct from list/dict; broadcast scalar with explicit `index`.

pd.Series: Examples

```
1 s = pd.Series([10, 20, 30], index=["a","b","c"], name="points")
2 s2 = pd.Series({"math": 88, "cs": 94, "stats": 90})
3 s3 = pd.Series(5, index=list("abcd"), dtype="int64")
```

Indexing a Series: .loc/.iloc/.at/.iat

Purpose: access by label or position; scalar fast paths.
.loc[key]: label access. **.iloc[i]**: positional access.
.at[label]: fast scalar by label. **.iat[i]**: fast scalar by position.
Slicing semantics:

- `s["a":"c"]` (labels) is *inclusive* of end.
- `s.iloc[0:2]` (positions) is *exclusive* of end.

Indexing a Series: Examples

```
s["b"]; s.loc["c"]; s.iloc[0]
s[["a","c"]]; s.iloc[[0,2]]

s["a":"c"]      # inclusive
s.iloc[0:2]     # exclusive

s.at["a"]       # scalar by label
s.iat[1]        # scalar by position
```

Series Utilities: Common Methods

Purpose: quick inspection stats.
Key methods and parameters:

- `head(n=5)`, `tail(n=5)`
- `describe(percentiles=None, include=None, exclude=None)`
- Comparisons: `s > 0` returns boolean Series usable for filtering.

Series Utilities: Examples

```
s.head(2); s.tail(1)
s.values, s.index, s.name
s.describe()
mask = s > 15
s[mask]
```

pd.to_datetime: Parameters & Usage

Purpose: parse strings/numbers to timestamps.
Signature (common): `pd.to_datetime(arg, format=None, utc=False, dayfirst=False, unit=None)`
Parameters:

- `arg`: scalar/list/Series/Index of date-like values.
- `format`: strptime format string for speed/strictness (e.g., `"%Y-%m-%d"`).
- `utc`: return UTC timezone-aware timestamps.
- `unit`: for epoch values (e.g., `"s"`, `"ms"`, `"ns"`).
- `dayfirst`: interpret dates like `02/03/2025` as D/M/Y.

pd.to_datetime: Examples

```
pd.to_datetime(["2025-07-01", "2025/07/02", "July 3, 2025"])
pd.to_datetime(["01-02-2025","02-03-2025"], dayfirst=True, format="%d-%m-%Y")
```

pd.date_range: Parameters & Usage

Purpose: generate fixed-frequency DatetimeIndex.
Signature (common): `pd.date_range(start=None, end=None, periods=None, freq=None, tz=None)`
Parameters:

- One of: `(start, periods)` or `(end, periods)` or `(start, end)`.
- `freq`: e.g., `"D"`, `"H"`, `"MS"`.
- `tz`: timezone string (e.g., `"US/Central"`).

pd.date_range: Examples

```
rng = pd.date_range(start="2025-01-01", periods=10, freq="B")
rng_tz = pd.date_range("2025-02-01", "2025-02-03", freq="H", tz="US/Central")
```

pd.DataFrame: Constructors

Purpose: 2D labeled table.

Common constructors:

- `pd.DataFrame(data, columns=None, index=None, dtype=None)`
- `pd.DataFrame.from_records(records, columns=None, index=None)`

Parameters:

- `data`: dict of lists/arrays, 2D ndarray, list of dicts, etc.
- `columns`: column labels/order; `index`: row labels.
- `dtype`: enforce a single dtype (or set per-column afterward).

pd.DataFrame: Examples

```
df = pd.DataFrame({
    "name": ["Alice", "Bob", "Cathy", "Dan"],
    "age":  [24, 29, 22, 31],
    "city": ["NYC", "LA", "NYC", "Chicago"]
})

records = [{"name":"Eva","age":28,"city":"Austin"},
           {"name":"Finn","age":35,"city":"LA"}]
df2 = pd.DataFrame.from_records(records, columns=["name","city","age"])

arr = np.arange(12).reshape(4, 3)
df3 = pd.DataFrame(arr, columns=["A","B","C"])
```

Inspecting: info/describe/dtypes

Purpose: schema + summary statistics.
Methods:

- `df.info(memory_usage=None, verbose=None)`
- `df.describe(percentiles=None, include=None, exclude=None)`
- `df.dtypes` (property)

Inspecting: Examples

```
df.shape, df.columns, df.index
df.info()
df.dtypes
df.describe()
```

Index Ops: set_index/reset_index/rename

Purpose: manage row labels and column names.
Methods:

- `df.set_index(keys, drop=True, inplace=False)`
- `df.reset_index(level=None, drop=False, inplace=False)`
- `df.rename(index=None, columns=None, inplace=False)`

Parameters: `drop=True` removes the column used as index; `inplace` writes back.

Index Ops: Examples

```
df_idx = df.set_index("name")
df_idx.loc["Alice"]
df_idx = df_idx.reset_index()

df_renamed = df.rename(columns={"city":"City"})
df_renamed.index.name = "row_id"
```

Selection: loc/iloc/boolean/query

Purpose: subset rows/cols by labels, positions, or conditions.
Key forms:

- `df.loc[row_labels, col_labels]`
- `df.iloc[row_idx, col_idx]`
- `df[cond]` or `df.query("expr")`

Helpers: `isin(values)`, comparisons (`>`, `>=`, `==`), bitwise `&`, `|`, `~`.

Selection: Examples

```
df["age"]; df[["name","city"]]
df.loc[0:2, ["name","age"]]   # label slice inclusive
df.iloc[0:3, 0:2]             # position slice exclusive
df[df["city"]=="NYC"]
df[(df["age"]>=25) & (df["city"].isin(["LA","NYC"]))]
```

Scalars, Compare, where

Purpose: fast cell access and conditional replacement.
Methods:

- `df.at[row_label, col_label]`, `df.iat[i, j]`
- `Series.where(cond, other=np.nan, inplace=False)`

Scalars, Compare, where: Examples

```
df.at[0,"age"]; df.iat[1,2]
cmp = df["age"] > df["age"].mean()
df_filtered = df[cmp]
df["age_adj"] = df["age"].where(df["age"] >= 25, other=np.nan)
```

Types: astype/to_numeric/category

Purpose: enforce/correct dtypes and memory use.
Methods:

- `Series.astype(dtype)` where dtype like `"int64"`, `"float64"`, `"category"`.
- `pd.to_numeric(obj, errors="raise"|"coerce"|"ignore", downcast=None)`
- Categorical: `Series.astype("category")` for repeated strings.

Types: Examples

```
df["age"] = df["age"].astype("int64")
pd.to_numeric(pd.Series(["10","x","20"]), errors="coerce")
df["city"] = df["city"].astype("category")
```

Statistics & Aggregations

Purpose: summarize series/columns and combine metrics.
Methods:

- `Series.mean()`, `median()`, `std()`, `min()`, `max()`
- `DataFrame.agg(funcs_per_col_dict)` e.g., `{"age": ["min","max","mean"]}`
- `sum(axis=0|1)` column/row-wise.

Statistics & Aggregations: Examples

```
df["age"].mean(), df["age"].median(), df["age"].std()
df.agg({"age": ["min", "max", "mean"]})
df3.sum(axis=0); df3.sum(axis=1)
```

Sort: sort_values/sort_index

Purpose: order rows/columns deterministically.
Methods:

- `df.sort_values(by, ascending=True, na_position="last", key=None)`
- `df.sort_index(axis=0, ascending=True)`

Sort: Examples

```
df.sort_values(by=["city","age"], ascending=[True, False])
df.sort_index()
df.sort_index(axis=1)
```

Apply family: map/apply/applymap

Purpose: flexible function application.
Differences:

- `Series.map(func|dict)` element-wise on a Series.
- `Series.apply(func)` element-wise (Series in, scalar or Series out).
- `DataFrame.apply(func, axis=0|1)` per column or per row.

Apply family: Examples

```
df["name_len"] = df["name"].map(len)
def age_bucket(x): return "Y" if x < 25 else "A"
df["age_bucket"] = df["age"].apply(age_bucket)
df_num = df3.copy()
```

Add/Remove/Concat: insert/drop/concat

Purpose: manage columns/rows and combine tables.
Methods:

- `df.insert(loc, column, value, allow_duplicates=False)`
- `df.drop(labels=None, axis=0|1, columns=None, index=None)`
- `pd.concat(objs, axis=0|1, ignore_index=False, join="outer")`

Add/Remove/Concat: Examples

```
df["score"] = [90, 82, 88, 95]
df.insert(1, "initials", df["name"].str[:1])
df_dropcol = df.drop(columns=["score"])
df_droprows = df.drop(index=[1, 3])
bigger = pd.concat([df, df2], ignore_index=True)
```

Series

Do:

1. Create a Series of 5 integers with labels `"p"...."t"`. Print first/last using `.iat`.
2. Build a mask for values $\geq$ mean and filter the Series.
3. Replace values $<$ median with `NaN` via `where`.

Datetime

Do:

1. pd.date_range for April 2025 (10 business days). Show only month-ends.
2. Parse `["2025-04-01", "Apr 2, 2025", "bad"]` with `errors="coerce"` and count NaT.

DataFrame Selection

Do:

1. Build `df` with `["product","price","city"]` (4 rows). Select `["product","city"]`.
2. Rows 1..2 by `.loc` and `.iloc`.
3. Add `price_usd = price * 1.1`; sort by `city` asc, then `price_usd` desc.

Mini Cheat Sheet

Task	Method(s)
Select columns	`df["col"]`, `df[["c1","c2"]]`
Rows by label/pos	`df.loc[.., ..]`, `df.iloc[.., ..]`
Single cell	`df.at[row, col]`, `df.iat[i,j]`
Filter	`df[cond]`, `df.query("expr")`
Sort	`sort_values`, `sort_index`
Types	`astype`, `to_numeric`, `"category"`
Datetime	`to_datetime`, `date_range`
Mutate	`assign`, `map`, `transform`
Add/Remove	`insert`, `drop`, `concat`